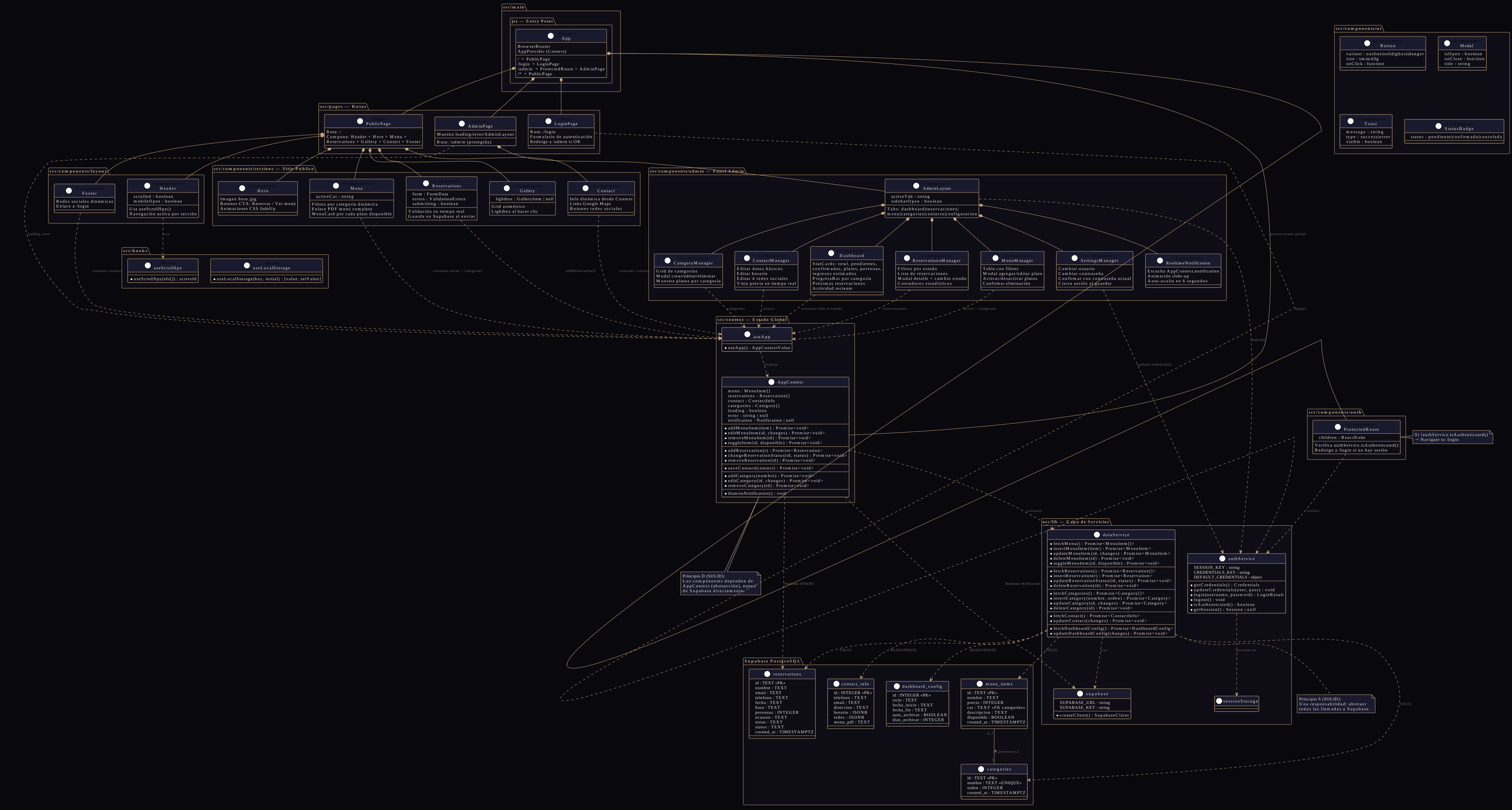
src/main
jsx — Entry Point
App
BrowserRouter
AppProvider (Context)
/ → PublicPage
/login → LoginPage
/admin → ProtectedRoute > AdminPage
/* → PublicPage
src/pages — Rutas
PublicPage
Ruta: /
Compone: Header + Hero + Menu + Reservations + Gallery + Contact + Footer
AdminPage
Muestra loading/error/AdminLayout
Ruta: /admin (protegida)
LoginPage
Ruta: /login
Formulario de autenticación
Redirige a /admin si OK
src/components/ui
Button
variant : outline|solid|ghost|danger
size : sm|md|lg
onClick : function
Modal
isOpen : boolean
onClose : function
title : string
Toast
message : string
type : success|error
visible : boolean
StatusBadge
status : pendiente|confirmada|cancelada
src/components/layout
Footer
Redes sociales dinámicas
Enlace a /login
Header
scrolled : boolean
mobileOpen : boolean
Usa useScrollSpy()
Navegación activa por sección
src/components/sections — Sitio Público
Hero
Imagen hero.jpg
Botones CTA: Reservar / Ver menú
Animaciones CSS fadeUp
Menu
activeCat : string
Filtros por categoría dinámica
Enlace PDF menú completo
MenuCard por cada plato disponible
Reservations
form : FormData
errors : ValidationErrors
submitting : boolean
Validación en tiempo real
Guarda en Supabase al enviar
Gallery
lightbox : GalleryItem | null
Grid asimétrico
Lightbox al hacer clic
Contact
Info dinámica desde Context
Links Google Maps
Botones redes sociales
src/components/admin — Panel Admin
AdminLayout
activeTab : string
sidebarOpen : boolean
Tabs: dashboard|reservaciones|menu|categorias|contacto|configuracion
CategoryManager
Grid de categorías
Modal crear/editar/eliminar
Muestra platos por categoría
ContactManager
Editar datos básicos
Editar horario
Editar 4 redes sociales
Vista previa en tiempo real
Dashboard
StatCards: total, pendientes, confirmadas, platos, personas, ingresos estimados
ProgressBar por categoría
Próximas reservaciones
Actividad reciente
ReservationsManager
Filtros por estado
Lista de reservaciones
Modal detalle + cambio estado
Contadores estadísticos
MenuManager
Tabla con filtros
Modal agregar/editar plato
Activar/desactivar platos
Confirmar eliminación
SettingsManager
Cambiar usuario
Cambiar contraseña
Confirmar con cogerContraseña actual
Cierra sesión al guardar
RealtimeNotification
Escucha AppContext.notification
Animación slide-up
Auto-oculta en 6 segundos
src/hooks
useScrollSpy
useScrollSpy(ids[]) : activeId
useLocalStorage
useLocalStorage(key, initial) : [value, setValue]
src/context — Estado Global
useApp
useApp() : AppContextValue
AppContext
menu : MenuItem[]
reservations : Reservation[]
contact : ContactInfo
categories : Category[]
loading : boolean
error : string | null
notification : Notification | null
addMenuItem(item) : Promise<void>
editMenuItem(id, changes) : Promise<void>
removeMenuItem(id) : Promise<void>
toggleItem(id, disponible) : Promise<void>
addReservation(r) : Promise<Reservation>
changeReservationStatus(id, status) : Promise<void>
removeReservation(id) : Promise<void>
saveContact(contact) : Promise<void>
addCategory(nombre) : Promise<void>
editCategory(id, changes) : Promise<void>
removeCategory(id) : Promise<void>
dismissNotification() : void
src/components/auth
ProtectedRoute
children : ReactNode
Verifica authService.isAuthenticated()
Redirige a /login si no hay sesión
Si !authService.isAuthenticated() → Navigate to /login
src/lib — Capa de Servicios
dataService
fetchMenu() : Promise<MenuItem[]>
insertMenuItem(item) : Promise<MenuItem>
updateMenuItem(id, changes) : Promise<MenuItem>
deleteMenuItem(id) : Promise<void>
toggleMenuItem(id, disponible) : Promise<void>
fetchReservations() : Promise<Reservation[]>
insertReservation(r) : Promise<Reservation>
updateReservationStatus(id, status) : Promise<void>
deleteReservation(id) : Promise<void>
fetchCategories() : Promise<Category[]>
insertCategory(nombre, orden) : Promise<Category>
updateCategory(id, changes) : Promise<Category>
deleteCategory(id) : Promise<void>
fetchContact() : Promise<ContactInfo>
updateContact(changes) : Promise<void>
fetchDashboardConfig() : Promise<DashboardConfig>
updateDashboardConfig(changes) : Promise<void>
authService
SESSION_KEY : string
CREDENTIALS_KEY : string
DEFAULT_CREDENTIALS : object
getCredentials() : Credentials
updateCredentials(user, pass) : void
login(username, password) : LoginResult
logout() : void
isAuthenticated() : boolean
getSession() : Session | null
supabase
SUPABASE_URL : string
SUPABASE_KEY : string
createClient() : SupabaseClient
sessionStorage
Principio S (SOLID):
Una responsabilidad: abstraer todas las llamadas a Supabase.
Principio D (SOLID):
Los componentes dependen de AppContext (abstracción), nunca de Supabase directamente.
Supabase PostgreSQL
reservations
id : TEXT «PK»
nombre : TEXT
email : TEXT
telefono : TEXT
fecha : TEXT
hora : TEXT
personas : INTEGER
ocasion : TEXT
notas : TEXT
status : TEXT
created_at : TIMESTAMPTZ
contact_info
id : INTEGER «PK»
telefono : TEXT
email : TEXT
direccion : TEXT
horario : JSONB
redes : JSONB
menu_pdf : TEXT
dashboard_config
id : INTEGER «PK»
ciclo : TEXT
fecha_inicio : TEXT
fecha_fin : TEXT
auto_archivar : BOOLEAN
dias_archivar : INTEGER
menu_items
id : TEXT «PK»
nombre : TEXT
precio : INTEGER
cat : TEXT «FK categories»
descripcion : TEXT
disponible : BOOLEAN
created_at : TIMESTAMPTZ
categories
id : TEXT «PK»
nombre : TEXT «UNIQUE»
orden : INTEGER
created_at : TIMESTAMPTZ
pertenece a
0..*
loading, error
usa
consume contact
consume menu + categories
addReservation()
consume contact
provee estado global
consume todo el estado
categories
contact
reservaciones
menu + categories
login()
logout()
updateCredentials()
expone
consume
verifica
consume INSERT
Realtime WebSocket
CRUD
READ/UPDATE
READ/UPDATE
CRUD
usa
persiste en
CRUD